

Data Structures Lab (24CS203P)

Experiment 7

Applications of stack

Name: Yash Vardhan singh**Semester:** 3**Roll No.:** 25BCP251**Faculty:** Dr. Davinder Paul Singh**Division:** 5 (G10)**Lab:** 5**Branch:** Computer Engineering

Program 1

Aim: Write a program to evaluate the following given postfix expressions:

a. 2 3 1 * + 9 -

Output: -4

b. 2 2 + 2 / 5 * 7 +

Output: 17

Code:

```
#include <stdio.h>
```

```
#include <conio.h>
```

```
#include <ctype.h>
```

```
#define MAX 100
```

```
int stack[MAX];
```

```
int top = -1;
```

```
void push(int value)
```

```
{
```

```
    top++;
```

```
    stack[top] = value;
```

```
}
```

```
int pop()
{
    int value;
    value = stack[top];
    top--;
    return value;
}

int evaluate(char exp[])
{
    int i;
    int a, b, result;

    for(i = 0; exp[i] != '\0'; i++)
    {
        if(exp[i] >= '0' && exp[i] <= '9')
        {
            push(exp[i] - '0');
        }
        else if(exp[i] == ' ')
        {
            continue;
        }
        else
        {
            b = pop();
            a = pop();

            switch(exp[i])
            {
                case '+':
```

```
        result = a + b;
        break;

    case '-':
        result = a - b;
        break;

    case '*':
        result = a * b;
        break;

    case '/':
        result = a / b;
        break;
    }

    push(result);
}

return pop();
}

void main()
{
    char exp1[] = "2 3 1 * + 9 -";
    char exp2[] = "2 2 + 2 / 5 * 7 +";

    clrscr();

    printf("Postfix Expression 1: %s", exp1);
    printf("\nOutput = %d", evaluate(exp1));
```

```
top = -1;

printf("\n\nPostfix Expression 2: %s", exp2);
printf("\nOutput = %d", evaluate(exp2));

getch();
}
```

Output:A screenshot of a terminal window with a black background and white text. The text shows the output of a program, including two postfix expressions and their evaluated results.

```
postfix expression 1 : 231*+9-
output = -4
postfix exp 2: 22+2/5*7+
output = 17
```

Program 2

Aim: Convert the given infix expression into postfix expression using stack.

Example- Input: $a + b * (c^d - e)^{(f + g * h)} - i$

Output: $abcd^e - fgh * +^* + i -$

Code:

```
#include <stdio.h>
```

```
#include <conio.h>
```

```
#include <ctype.h>
```

```
#include <string.h>
```

```
#define MAX 100
```

```
char stack[MAX];
```

```
int top = -1;
```

```
void push(char ch)
```

```
{
```

```
    top++;
```

```
    stack[top] = ch;
```

```
}
```

```
char pop()
```

```
{
```

```
    char ch;
```

```
    ch = stack[top];
```

```
    top--;
```

```
    return ch;
```

```
}
```

```
int precedence(char ch)
```

```
{
```

```
    if(ch == '^')
        return 3;
    else if(ch == '*' || ch == '/')
        return 2;
    else if(ch == '+' || ch == '-')
        return 1;
    else
        return 0;
}

void infixToPostfix(char infix[], char postfix[])
{
    int i, j = 0;
    char ch;

    for(i = 0; infix[i] != '\0'; i++)
    {
        ch = infix[i];

        if(isalnum(ch))
        {
            postfix[j] = ch;
            j++;
        }
        else if(ch == '(')
        {
            push(ch);
        }
        else if(ch == ')')
        {
            while(top != -1 && stack[top] != '(')
            {
```

```
        postfix[j] = pop();
        j++;
    }

    pop();
}
else
{
    while(top != -1 &&
        stack[top] != '(' &&
        precedence(stack[top]) >= precedence(ch))
    {
        postfix[j] = pop();
        j++;
    }

    push(ch);
}

while(top != -1)
{
    postfix[j] = pop();
    j++;
}

postfix[j] = '\0';
}

void main()
{
    char infix[MAX];
```

```
char postfix[MAX];

clrscr();

printf("Enter infix expression: ");
gets(infix);

infixToPostfix(infix, postfix);

printf("\nPostfix expression: %s", postfix);

getch();
}
```

Output:

```
infix: a+b*(c^a-d-e)^(f+g*h)-i
postfix: abcd^e-fgh*+^**+i-
=
```

Program 3

Aim : Given an expression, write a program to examine whether the pairs and the orders of “{”, “}”, “(”, “)”, “[”, “]” are correct in the expression or not.

Example: Input: exp = “[()] { } { [() () () } ”

Output: Balanced

Input: exp = “[(] ”

Output: Not Balanced

Code :

```
#include <stdio.h>
```

```
#include <conio.h>
```

```
#define MAX 100
```

```
char stack[MAX];
```

```
int top = -1;
```

```
void push(char ch)
```

```
{
```

```
    top++;
```

```
    stack[top] = ch;
```

```
}
```

```
char pop()
```

```
{
```

```
    char ch;
```

```
    ch = stack[top];
```

```
    top--;
```

```
    return ch;
```

```
}
```

```
int isMatching(char open, char close)
```

```
{
```

```
    if(open == '(' && close == ')')
```

```
        return 1;

    if(open == '[' && close == ']')
        return 1;

    if(open == '{' && close == '}')
        return 1;

    return 0;
}

int isBalanced(char exp[])
{
    int i;
    char ch;

    top = -1;

    for(i = 0; exp[i] != '\0'; i++)
    {
        ch = exp[i];

        if(ch == '(' || ch == '[' || ch == '{')
        {
            push(ch);
        }
        else if(ch == ')' || ch == ']' || ch == '}')
        {
            if(top == -1)
                return 0;

            if(!isMatching(pop(), ch))
```

```
        return 0;
    }
}

if(top == -1)
    return 1;

return 0;
}

void main()
{
    char exp[MAX];

    clrscr();

    printf("Enter expression: ");
    gets(exp);

    if(isBalanced(exp))
        printf("\nOutput: Balanced");
    else
        printf("\nOutput: Not Balanced");

    getch();
}
```

Output :

```
exp = "[()]{ }{ [()() ]() }" -> Balanced  
exp = "[()]" -> Not Balanced  
-
```